

TESS Architecture and Logic Analysis

Generated from repository analysis

TESS Architecture and Logic Analysis

1. Executive Summary

TESS stands for Terminal Embedded System Supervisor. In this repository it behaves as a terminal-first AI operating layer that combines:

- an LLM-driven planner and command generator,
- a guarded action dispatcher,
- a set of concrete automation modules for Windows, the browser, files, voice, WhatsApp, YouTube, and PDFs,
- persistent personalization and memory,
- optional web and API frontends.

The core architecture is not a single monolithic agent. It is a pipeline:

1. Input enters through a CLI, coding REPL, API endpoint, or wake listener.
2. A ``Brain`` instance builds context and asks an LLM for a strict JSON action.
3. ``AgenticLoop`` optionally turns one user request into several execution steps.
4. ``SecurityEngine`` validates dangerous actions.
5. ``ActionDispatcher`` routes the action to the relevant subsystem.
6. Specialized components perform the real work.
7. Memory, profile, and history are updated for future interactions.

This means TESS is best described as an LLM-centered orchestrator wrapped around deterministic tool modules.

2. Repository Structure

The most important paths are:

- ``tess_cli/__main__.py``: package entry point.
- ``tess_cli/cli.py``: terminal bootstrap and interactive loop.
- ``tess_cli/core/brain.py``: LLM interface, context enrichment, retry logic, JSON parsing.
- ``tess_cli/core/agent_loop.py``: multi-step planning and execution loop.
- ``tess_cli/core/orchestrator.py``: central action dispatcher.
- ``tess_cli/core/config.py``: configuration, module toggles, and system prompt construction.
- ``tess_cli/core/security.py``: safety checks.
- ``tess_cli/core/executor.py``: shell execution with timeout.
- ``tess_cli/core/memory_engine.py``: JSON-backed memory.
- ``tess_cli/core/knowledge_base.py``: ChromaDB-backed semantic memory with fallback.
- ``tess_cli/api/server.py``: FastAPI interface and live session API.
- ``tess_cli/core/voice_client.py``: recording, smart listening, system-audio capture, transcription, speech.
- ``tess_cli/scripts/wake_listener.py``: clap-plus-wake-word launcher.
- ``tess_cli/skills/pdf_skill.py``: PDF operations.
- ``tess_cli/core/coding_agent.py`` and ``tess_cli/core/coding_engine.py``: coding-mode subsystem.

TESS Architecture and Logic Analysis

Generated from repository analysis

3. Startup Paths

3.1 Main Package Entry

``tess_cli/__main__.py`` suppresses warnings early, imports ``main`` from ``tess_cli.cli``, and runs it.

This means the package entry is intentionally thin. Real startup logic lives in ``cli.py``.

3.2 CLI Entry Modes

``tess_cli/cli.py`` supports multiple fast-exit modes before the normal terminal loop:

- ``tess init``: launches the setup wizard.
- ``tess build [path]``: launches the Ralph builder loop.
- ``tess code [path]``: enters coding mode.
- ``tess eval``: runs the regression harness.
- ``tess``: starts the main terminal assistant.

So TESS is not one mode. It is a family of modes sharing the same underlying components.

3.3 API Supervisor

``supervisor.py`` runs ``python -m tess_cli.api.server`` in a restart loop. If the API exits with a non-zero code, the supervisor sleeps 3 seconds and restarts it.

This is a simple availability strategy:

- normal exit code ``0`` means stop,
- any other exit code means restart.

3.4 Wake Listener

``tess_cli/scripts/wake_listener.py`` continuously listens to the microphone, detects two claps inside a time window, then records a short phrase and uses Whisper to check for the word ``tess``. If confirmed, it launches ``Start_TESS.bat``.

This makes TESS bootable without typing, but the listener itself is still deterministic and threshold-based.

4. Main Runtime Boot Sequence

When normal CLI mode starts, ``main()`` in ``tess_cli/cli.py`` performs the following initialization steps:

1. Create ``UserProfile``.
2. Create ``KnowledgeBase`` if memory is enabled.
3. Create ``ProfileManager``.
4. Create ``Executor``.
5. Create ``SecurityEngine``.
6. Get a ``Brain`` instance for the terminal user.
7. Set the brain personality and system prompt.

TESS Architecture and Logic Analysis

Generated from repository analysis

8. Register core components into a shared ``comps`` dictionary.
9. Conditionally add optional modules depending on config.
10. Load plugin skills dynamically.
11. Start background services like librarian, scheduler, telegram bot, and terminal visibility hotkey.
12. Enter the main input loop.

The shared ``comps`` dictionary is an important design choice. Most of TESS acts like a dependency-injected runtime graph where components are looked up by string key instead of direct hard references.

5. Core Execution Model

5.1 Human Request to Machine Action

The canonical flow is:

1. User types a message.
2. ``AgenticLoop.run()`` receives the message.
3. It appends an extra system instruction telling the model to behave as an agent-planner.
4. ``Brain.generate_command()`` creates a JSON action.
5. ``SecurityEngine.validate_action()`` checks safety.
6. ``process_action()`` and ``ActionDispatcher.dispatch()`` route the action.
7. A concrete subsystem executes the work.
8. The result is reflected back into the loop.
9. The loop either ends or asks the LLM for a next step.

This is a classic perceive-plan-act-reflect loop, but in a lightweight local implementation.

5.2 AgenticLoop State Machine

``AgenticLoop`` has:

- ``max_steps = 10``
- ``max_replans = 3``

The loop logic is:

1. Ask for one action.
2. Validate payload shape.
3. Run security.
4. Execute the action.
5. Inspect the result for failure markers.
6. If the result looks bad, add a reflection and replan.
7. If the action is terminal, stop.

The failure heuristic is string-based. A result is considered a failure if it contains any of:

- ``error``

TESS Architecture and Logic Analysis

Generated from repository analysis

- `failed`
- `timed out`
- `unknown`
- `not found`
- `blocked`
- `unavailable`
- `disabled`

This is not mathematically deep, but it is an important control heuristic. It means TESS sometimes judges success through keyword classification instead of structured result types.

6. The Brain: LLM Control Layer

6.1 Responsibility

`Brain` is the translation layer between natural language and structured tool actions. It does four main things:

- builds the prompt and context,
- calls the selected model provider,
- recovers structured JSON from imperfect model output,
- keeps short-term history and long-term memory connected.

6.2 Provider Factory

`LLMClientFactory.get\_client()` supports:

- Groq,
- OpenAI,
- DeepSeek through OpenAI-compatible API,
- Gemini.

The active provider and model come from `Config`.

6.3 Retry and Backoff Math

The method `\_call\_api\_with\_retry()` uses up to `max\_retries = 5`.

For rate-limit style failures, the sleep time is:

$\text{sleep_time} = (2^{** \text{attempt}}) + \text{random.uniform}(0, 1)$

If attempt numbering starts at 0, the nominal waits are approximately:

- attempt 0: `1 to 2` seconds,
- attempt 1: `2 to 3` seconds,
- attempt 2: `4 to 5` seconds,
- attempt 3: `8 to 9` seconds,
- attempt 4: `16 to 17` seconds.

TESS Architecture and Logic Analysis

Generated from repository analysis

This is exponential backoff with additive random jitter. The logic matters because it prevents synchronized repeated failures and rotates API keys after rate limits.

6.4 API Key Rotation Logic

When a retryable error occurs, `current_key_index += 1`.

`Config.get_api_key(provider, index)` returns:

`keys[index % len(keys)]`

That is modular arithmetic. If there are `n` keys, key usage cycles through indices:

`0, 1, 2, ..., n - 1, 0, 1, ...`

This guarantees wraparound instead of an out-of-range failure.

6.5 JSON Recovery Logic

The model is expected to return one JSON object, but `_parse_json()` handles imperfect output in stages:

1. strip markdown code fences,
2. try `raw_decode`,
3. try regex extraction,
4. use brace counting to find a balanced JSON object,
5. fall back to a `reply_op`.

The brace-counting algorithm is:

- initialize `brace_count = 0`,
- increment for every `{`,
- decrement for every `}`,
- when `brace_count == 0`, the object boundary is considered closed.

This is a small parser-style algorithm used to tolerate malformed or wrapped model output.

6.6 Context Distillation

If history length reaches 100 messages, `_maybe_distill_context()` asks the model for a concise summary and replaces most of the history with:

- the original system prompt,
- one summary system message,
- the last 8 messages.

This is a bounded-context compression strategy.

6.7 Context Enrichment

Before generation, `_enrich_context()` optionally adds:

TESS Architecture and Logic Analysis

Generated from repository analysis

- profile facts,
- one top memory search result,
- vault key awareness.

This means the final prompt is not only the raw chat history. It is history plus retrieved context.

7. System Prompt and Action Schema Logic

``Config.get_system_prompt()`` is where TESS behavior is heavily defined. The prompt encodes:

- identity,
- operating environment,
- Windows shell constraints,
- JSON-only output requirements,
- action schema,
- tool selection rules,
- anti-hallucination rules.

This prompt is not decoration. It is effectively part of the program logic because it constrains the LLM to produce a narrow action format.

The prompt also encodes several policy shortcuts:

- music requests must use ``youtube_op``,
- WhatsApp tasks must use ``whatsapp_op``,
- Windows commands should use PowerShell syntax,
- simulation requests must use ``experimental_op``.

So some "logic" exists in Python and some exists in prompt engineering.

8. Action Dispatch Layer

8.1 Dispatcher Design

``ActionDispatcher.dispatch()`` works in two phases:

1. If the action name matches a dynamically loaded skill, run that skill.
2. Otherwise call a method named ``_handle_<action>``.

This is dynamic dispatch by naming convention.

8.2 Why This Matters

The architecture allows TESS to grow in two ways:

- new hardcoded handlers in Python,
- new plugin skills without touching the core dispatcher.

TESS Architecture and Logic Analysis

Generated from repository analysis

That is a practical extensibility design.

9. Configuration Logic

9.1 Config Sources

Config is loaded from:

1. `~/tess/config.json``,
2. environment variables as fallback,
3. built-in defaults.

9.2 Deep Merge Logic

Nested dictionaries are merged recursively by `deep_update(d, u)``.

This means user configuration only overrides provided keys and inherits the rest. It avoids replacing the entire config tree.

9.3 Module Gating

Most subsystems are toggled by config booleans such as:

- `web_search``,
- `media``,
- `whatsapp``,
- `memory``,
- `planner``,
- `coding``.

This affects startup, which affects what actions are actually executable at runtime.

10. Safety Model

10.1 SecurityEngine Logic

`SecurityEngine`` applies three main categories of rules:

- regex blacklists for commands,
- sensitive path restrictions,
- high-security write blocking.

Examples of blocked patterns include:

- `rm -rf``
- `del /s``
- `format c:``
- `rd /s``

TESS Architecture and Logic Analysis

Generated from repository analysis

- `shutdown`
- `reg delete`

For file operations it blocks writes and deletes against sensitive paths like:

- `C:\Windows`
- `C:\Program Files`
- `AppData`
- `System32`

10.2 Important Limitation

The security model is heuristic, not formal. It does not prove safety. It blocks known-dangerous patterns.

11. Command Execution Logic

`Executor.execute\_command()`:

- optionally asks for confirmation in safe mode,
- forces PowerShell for consistency when needed,
- runs with a 60-second timeout,
- returns stdout plus stderr.

The timeout is a hard stop:

- success path: process exits before 60 seconds,
- failure path: `TimeoutExpired` returns a timeout error string.

This is a deterministic guard against hung commands.

12. File System Logic

`FileManager` implements:

- directory listing,
- bounded file reading,
- full writes,
- text replacement patching.

The `read\_file()` method defaults to `max\_lines = 500`. If a file is longer, it truncates output and reports the total line count. This is a context-window protection mechanism.

13. Memory and Retrieval Math

13.1 JSON Memory

`MemoryEngine` stores memories as JSON objects with:

TESS Architecture and Logic Analysis

Generated from repository analysis

- `id`,
- `timestamp`,
- `text`,
- `metadata`.

13.2 Retrieval Formula

`retrieve_context()` computes similarity between the query word set `Q` and memory word set `M` using Jaccard similarity:

$$\text{score} = |Q \cap M| / |Q \cup M|$$

Interpretation:

- numerator = shared words,
- denominator = total distinct words across both sets,
- larger score = stronger overlap.

This is a simple lexical similarity score, not an embedding-based semantic score.

13.3 KnowledgeBase Chunking Math

`KnowledgeBase._chunk_text(text, max_chars=1000, overlap=100)` slices long documents into overlapping chunks.

If:

- chunk size = `C`,
- overlap = `O`,

then the start position advances by:

$$\text{step} = C - O$$

With defaults:

- `C = 1000`,
- `O = 100`,
- `step = 900`.

In directory learning the code later uses:

- `max_chars = 1500`,
- `overlap = 150`,
- step becomes `1350`.

This overlap is important because it preserves context across chunk boundaries.

13.4 Vector Search

If ChromaDB is available, semantic search uses the collection's embedding function and `collection.query(...)`. If ChromaDB is unavailable, TESS falls back to the simpler Jaccard-based `MemoryEngine`.

TESS Architecture and Logic Analysis

Generated from repository analysis

So the repository contains two retrieval regimes:

- embedding-based semantic search,
- keyword-overlap fallback search.

14. User Profile Logic and Math

``UserProfile`` tracks facts, interests, and usage statistics.

14.1 Streak Calculation

On startup:

- parse last session date,
- compute ``diff = (today - last_session_date).days``

Then:

- if ``diff == 1``, increment streak,
- if ``diff > 1``, reset streak to 1,
- if same day, leave streak unchanged.

This is simple date-difference arithmetic.

14.2 Fact Extraction

User facts are detected through regex patterns like:

- ``my name is ...``
- ``i like ...``
- ``remember that ...``

It is pattern-based shallow information extraction rather than model-based NER.

15. Voice and Audio Math

15.1 Fixed Recording

``record_audio(duration, sample_rate)`` records:

``num_samples = duration * sample_rate``

For the default ``duration = 5`` and ``sample_rate = 44100``, the sample count is:

``5 * 44100 = 220500``

15.2 Smart Listening Threshold

``VoiceClient.listen()`` first records 0.5 seconds of ambient audio, then computes RMS noise:

TESS Architecture and Logic Analysis

Generated from repository analysis

```
`noise_rms = sqrt(mean(calib_data ** 2))`
```

Then the speech threshold is:

```
`threshold = max(noise_rms * 1.5, 300)`
```

This means:

- in a quiet room, threshold will never go below 300,
- in a noisy room, threshold scales with ambient energy.

15.3 Chunk Timing

Defaults:

- `sample\_rate = 16000`
- `chunk\_duration = 0.2` seconds
- `silence\_duration = 1.5` seconds
- `max\_duration = 30` seconds

Derived values:

- `chunk\_size = 16000 \* 0.2 = 3200` samples,
- `retention\_chunks = 1.5 / 0.2 = 7.5`, truncated to `7`,
- `max\_chunks = 30 / 0.2 = 150`.

The stop rule is:

- keep reading chunks,
- compute chunk energy ``sqrt(mean(data ** 2))``,
- increment ``silent_chunks`` when energy is below threshold,
- stop when ``silent_chunks > retention_chunks``.

This is a threshold-crossing finite-state detector.

15.4 System Audio Capture

For system audio:

```
`frames = duration * sample_rate`
```

and the float audio is converted to signed 16-bit PCM by:

```
`pcm = clip(data, -1.0, 1.0) * 32767`
```

then cast to ``int16``.

That is standard normalized-float to PCM conversion.

16. Wake Listener Math

TESS Architecture and Logic Analysis

Generated from repository analysis

The wake listener uses clap detection plus speech confirmation.

16.1 Calibration

It estimates RMS from 1 second of microphone input:

```
`rms = sqrt(mean(square(data)))`
```

Then the clap threshold is:

```
`threshold = max(rms * 4.0, 1200.0)`
```

Compared to normal speech detection, clap detection uses a much stronger multiplier because claps are high-energy impulses.

16.2 Event Timing

Defaults:

- `chunk\_duration = 0.05` seconds
- `clap\_window\_sec = 1.5`
- `clap\_refractory\_sec = 0.18`
- `post\_clap\_listen\_sec = 3.0`

Implications:

- audio is processed every 50 ms,
- two claps must happen inside a 1.5-second window,
- claps closer than 0.18 seconds are treated as the same clap,
- after two claps, TESS listens for up to 3 seconds for the wake word,
- after successful launch there is an 8-second cooldown.

This is event-window logic with a refractory period to avoid double counting.

17. Knowledge and Command Indexing Logic

`CommandIndexer` scans system commands, fetches help text, and indexes them into the knowledge base.

Notable quantitative limits:

- help fetch timeout: 3 seconds,
- help text truncation: first 2000 characters,
- progress printed every 20 indexed commands.

The logic is:

1. discover commands through PowerShell,
2. fetch help output,
3. scan nearby docs,
4. upsert into ChromaDB.

TESS Architecture and Logic Analysis

Generated from repository analysis

This turns OS commands into retrievable knowledge objects.

18. Browser, Desktop, Media, and Messaging Logic

18.1 DOM Automation

`DOMController` uses Playwright-backed page control with operations like:

- open,
- navigate,
- click,
- type,
- wait,
- extract text,
- screenshot.

Timeouts are mostly between 5 and 60 seconds depending on operation.

18.2 Desktop Vision

`DesktopVisionController` exposes a desktop-focused control layer:

- list visible apps,
- focus app,
- screenshot,
- click,
- type,
- hotkey,
- hide and show windows.

This lets TESS work beyond the browser.

18.3 YouTube Logic

`youtube\_op` is intentionally separated from generic web search. The orchestrator defaults to:

- `play` when there is a query but no sub-action,
- `stop` or `pause` for certain single-word stop commands.

This separation is repeated in the prompt, so both code and prompt try to keep music control distinct from research.

18.4 WhatsApp Logic

`whatsapp\_op` maps to:

- `send`,
- `monitor` or `chat`,
- `call`,

TESS Architecture and Logic Analysis

Generated from repository analysis

- `answer`,
- `stop`.

If the LLM forgets the sub-action, the dispatcher infers it from the text. That is another heuristic repair step.

19. API Server Logic

`tess\_cli/api/server.py` exposes:

- config endpoints,
- chat endpoint,
- session creation,
- live event feeds,
- upload and transcript flows.

It also contains a direct Gemini HTTP call with:

- `temperature = 0.4`
- `topP = 0.9`
- `maxOutputTokens = 1800`
- request timeout `45` seconds

This is effectively a second LLM access path outside the `Brain` abstraction.

19.1 Session Model

`SessionState` holds:

- `id`,
- `created\_at`,
- `mode`,
- `resume\_text`,
- `api\_key`,
- event history,
- subscriber queues.

This supports live phone-laptop sessions with server-sent style updates.

20. Coding Mode Logic

Coding mode is a specialized agent system separate from the main assistant loop.

20.1 Tool Loop

`CodingAgent` asks the model for one JSON tool call at a time. Safe tools run automatically, dangerous tools require permission. The loop limit is:

- `MAX\_AGENT\_STEPS = 25`

TESS Architecture and Logic Analysis

Generated from repository analysis

20.2 CodingEngine Functions

`CodingEngine` supports:

- scaffolding projects,
- writing files,
- executing Python files,
- running tests,
- generating fixes from tracebacks,
- grep-like search,
- file outlines,
- block replacement.

20.3 Search Limits

`grep\_search()` returns the first 50 matches and reports total match count if more exist. This is a result-capping strategy for model context control.

21. PDF Logic

`PDFSkill` supports:

- merge,
- split,
- extract text,
- replace text,
- create.

21.1 Page Index Math

Human page input is 1-indexed, but PyMuPDF is 0-indexed. So the conversion is:

``internal_page = user_page - 1``

For a range `start-end`, the code inserts:

- from `start - 1`
- to `end - 1`

21.2 Text Replacement Geometry

For each match rectangle:

1. redact the original region,
2. insert replacement text at `rect.bl - (0, 2)`,
3. use font size `11`.

This is geometric overlay rather than semantic PDF editing.

TESS Architecture and Logic Analysis

Generated from repository analysis

22. Sandbox Logic

The sandbox subsystem chooses among:

- Docker,
- restricted subprocess fallback.

22.1 Resource Guarding

Restricted subprocess monitoring checks:

- elapsed time against timeout,
- resident memory against RAM limit.

The default values are loaded from env:

- RAM limit default: `200 MB`
- timeout default: `15 seconds`

The loop polls every `0.1` seconds.

22.2 Docker Memory Limit

Docker mode uses:

- `mem\_limit = "128m"`

So the code applies explicit memory bounds in both containerized and fallback modes.

23. Plugin and Skill Loading Logic

`SkillLoader` scans:

- internal `tess\_cli/skills`,
- user `~/.tess/plugins`

For every Python file it imports the module, inspects for subclasses of `BaseSkill`, instantiates them, and registers their intents.

This means TESS has a plugin architecture that relies on:

- filesystem discovery,
- dynamic import,
- reflection over class inheritance.

24. Important Architectural Patterns

The project uses several recurring patterns:

24.1 Shared Component Registry

TESS Architecture and Logic Analysis

Generated from repository analysis

Most runtime objects live in a dictionary keyed by short names. This is flexible but weakly typed.

24.2 Prompt-as-Policy

Many operating rules are embedded in the system prompt, not only in Python.

24.3 Heuristic Recovery

Examples:

- JSON repair through brace counting,
- action-subtype inference,
- failure detection through keywords,
- audio thresholds from RMS heuristics.

24.4 Graceful Degradation

Examples:

- ChromaDB falls back to JSON memory,
- Telegram bot failures retry,
- missing optional modules disable features instead of crashing startup.

25. Concrete Logic Summary by Subsystem

25.1 Decision Logic

- LLM proposes action JSON.
- Python validates minimal shape.
- Security filters risky actions.
- Dispatcher maps action to tool.

25.2 Retrieval Logic

- profile facts are appended,
- one top memory result is retrieved,
- vector DB is used when available,
- Jaccard retrieval is fallback.

25.3 Temporal Logic

- CLI loop is unbounded until exit,
- agent loop is capped at 10 steps,
- coding loop is capped at 25 steps,
- API retries use exponential waits,
- wake listener uses fixed windows and refractory intervals.

TESS Architecture and Logic Analysis

Generated from repository analysis

25.4 Numeric Guardrails

- command timeout: 60 s,
- coding execution timeout: 30 s,
- sandbox timeout: 15 s default,
- browser navigation timeout: commonly 60 s,
- file-read truncation: 500 lines,
- search truncation: 50 matches,
- history distillation threshold: 100 messages.

26. Observed Weaknesses and Inconsistencies

These are relevant for understanding how TESS works because they affect correctness.

26.1 PDF Dispatcher Mismatch

``orchestrator.py`` calls ``pdf.handle_action(data)``, but ``PDFSkill`` exposes ``run(...)`` and does not define ``handle_action(...)``.

Result:

- PDF operations are likely broken in the current code unless another compatibility layer exists elsewhere.

26.2 Mixed Execution Policies

The README claims autonomous coding without repeated permission prompts, but ``Executor`` still prompts in safe mode with ``Run? (Y/n):``.

Result:

- behavior depends on config and subsystem, so autonomy is not uniform.

26.3 Logic Split Between Prompt and Code

Some hard rules exist only in the prompt. If model adherence weakens, Python-side enforcement may not fully compensate.

26.4 Weak Typing in Shared Components

String-keyed component lookup is convenient but makes refactoring and static validation harder.

27. Final Mental Model

TESS is best understood as a layered control system:

1. Interfaces:
 - terminal, API, wake listener, telegram.
2. Cognitive layer:

TESS Architecture and Logic Analysis

Generated from repository analysis

`Brain`, prompt rules, memory enrichment, JSON parsing.

3. Planning layer:

`AgenticLoop`, `Planner`, coding agent loop.

4. Safety layer:

`SecurityEngine`, timeouts, sandbox.

5. Execution layer:

dispatcher plus concrete modules.

6. Persistence layer:

config, user profile, memory, vector DB.

The math inside TESS is lightweight but important. It mostly appears as:

- similarity scoring,
- chunking arithmetic,
- retry timing,
- threshold detection,
- date-difference calculations,
- bounded-loop control,
- index conversion.

There is no large numerical model implemented locally beyond Whisper inference and embedding functions delegated to external libraries. The repository's own "math" is mostly heuristic systems engineering math.

28. Bottom Line

TESS works by turning free-form user language into structured JSON actions, validating them, dispatching them to concrete automation modules, and looping with short reflections until a task is complete.

Its intelligence comes from the LLM layer plus retrieved context.

Its reliability comes from:

- action validation,
- safety filters,
- retries,
- timeouts,
- bounded loops,
- deterministic tool handlers.

Its most important mathematical and logical mechanisms are:

- exponential backoff with jitter,
- modular key rotation,
- Jaccard similarity retrieval,
- overlapping chunk segmentation,
- RMS-based audio thresholding,

TESS Architecture and Logic Analysis

Generated from repository analysis

- date-difference streak tracking,
- step-limited planning loops,
- 1-index to 0-index page mapping.

That is the real operational core of this codebase.